Arbeitsphase 1: Methoden definieren und aufrufen

Aufgaben 1 bis 3: Erst planen, dann programmieren.

- Eine eigene Methode definieren und aufrufen
- Mehrere Aufrufe in die richtige Reihenfolge bringen
- Dieselbe Anleitung mehrfach verwenden

Schreiben Sie vor dem Ausführen die erwartete Ausgabe auf. Erklären Sie danach, wann der Methodenkörper ausgeführt wird und wo es anschließend weitergeht.

Aufgabe 1: Eigener Name ausgeben

Ziel: Eine Methode definieren und aufrufen.

1. Schreiben Sie `PrintMyName` ohne Parameter und ohne Rückgabewert.
2. Geben Sie darin Ihren vollständigen Namen als festen Text aus.
3. Rufen Sie die Methode im Hauptprogramm genau einmal auf.

Kontrolle: Der Name erscheint genau einmal.

Aufgabe 2: Mehrere Bausteine aufrufen

Ziel: Mehrere Methoden in einer festen Reihenfolge aufrufen.

1. Schreiben Sie `PrintHeader` und `PrintFooter`, jeweils ohne Parameter mit Rückgabetyp `void`.
2. `PrintHeader` gibt `Willkommen!` aus, `PrintFooter` gibt `Bis bald!` aus.
3. Erzeugen Sie im Hauptprogramm: `Willkommen!`, `Programminhalt`, `Bis bald!`.

Kontrolle: Die Ausgabe folgt der Reihenfolge der Aufrufe.

Aufgabe 3: Eine Anleitung mehrfach verwenden

Ziel: Wiederholte Anweisungen einmal definieren.

1. Schreiben Sie `PrintCourseInfo` ohne Parameter mit Rückgabetyp `void`.
2. Die Methode gibt zwei Zeilen aus: `Informationstechnologie I` und `Eigene Methoden`.
3. Rufen Sie sie auf, geben Sie im Hauptprogramm `Pause` aus und rufen Sie sie erneut auf.

Kontrolle: Die beiden Aufrufe geben jeweils dieselben zwei Zeilen aus. Dazwischen steht `Pause`. Insgesamt erscheinen fünf Zeilen.

Arbeitsphase 2: Eingaben, Ergebnisse und Datenfluss

Aufgaben 4 bis 6: Methoden schreiben, verbinden und erklären.

- Parameter und Rückgabewerte nutzen
- Eine Prüfung und die Textausgabe zusammensetzen
- Wertkopie, Zuweisung und Gültigkeitsbereich nachvollziehen

Nutzen Sie weiterhin eigene Definitionen und Aufrufe. Trennen Sie Berechnung und Konsolenausgabe und prüfen Sie mehrere Eingabewerte.

Aufgabe 4: Begrüßung zurückgeben

Ziel: Einen Parameter mit einem Rückgabewert kombinieren.

1. Schreiben Sie `GetGreeting` mit dem Parameter `string name` und dem Rückgabetyp `string`.
2. Geben Sie den Text `"Hallo, " + name + "!"` zurück.
3. Rufen Sie die Methode mit Ihrem Namen auf.
4. Speichern Sie das Ergebnis und geben Sie es im Hauptprogramm aus.

Test: `GetGreeting("Mia")` liefert `"Hallo, Mia!"`.

Aufgabe 5: Gerade oder ungerade?

Ziel: Eine Prüfung und eine Textbestimmung verbinden.

1. Schreiben Sie `bool IsEven(int number)`. Prüfen Sie, ob `number` ohne Rest durch `2` teilbar ist.
2. Verwenden Sie das Ergebnis zunächst in `if`/`else`, um `Gerade` oder `Ungerade` auszugeben.
3. Lagern Sie die Textbestimmung in `string GetParityText(bool even)` aus. Die Methode gibt den passenden Text zurück.
4. Verbinden Sie beide Methoden im Hauptprogramm. Geben Sie dort den zurückgegebenen Text aus.

Tests: `8` und `0` ergeben `Gerade`, `7` ergibt `Ungerade`.

Tipp: `%` liefert den Divisionsrest.

Aufgabe 6: Werte und Aufrufe nachvollziehen

Ziel: Wertkopie, Rückgabe und Zuweisung unterscheiden.

```
int number = 3;
int first = DoubleValue(number);
DoubleValue(number);
number = DoubleValue(number);
int DoubleValue(int value)
{
    value = value * 2;
    return value;
}
```

Welche Werte haben `number` und `first` nach jedem Aufruf? Geben Sie beide am Ende aus. Erklären Sie Zeile 3 und den Gültigkeitsbereich von `value`.

Arbeitsphase 3: Übergabearten und Rekursion

Aufgaben 7 bis 10: Die bisherigen Grundlagen gemeinsam einsetzen.

- Variablen mit `ref` verändern und Ergebnisse mit `out` liefern

- Mit `TryParse`, `bool` und `in` arbeiten
- Eine rekursive Methode schreiben und ihre Rückgaben erklären

Planen Sie jeweils Eingaben, Verarbeitung und Ausgabe. Verwenden Sie passende Tests und erklären Sie den Datenfluss bis ins Hauptprogramm.

Aufgabe 7: Texte mit `ref` tauschen

Ziel: Zwei Variablen im Hauptprogramm durch eine Methode ändern.

1. Schreiben Sie `Swap` mit zwei `ref string`-Parametern.
2. Tauschen Sie die Werte mithilfe einer temporären Variable.
3. Legen Sie im Hauptprogramm zwei Texte an und geben Sie sie aus.
4. Rufen Sie `Swap` auf und geben Sie beide Variablen erneut aus.

Test: Vor dem Aufruf enthalten die Variablen `"links"` und `"rechts"`. Danach enthalten sie `"rechts"` und `"links"`.

Aufgabe 8: Kreisumfang und Kreisfläche

Ziel: Zwei Ergebnisse mit `out` liefern.

1. Schreiben Sie `GetCircleDetails` mit `double radius`, `out double circumference` und `out double area`; Rückgabetyp `void`.
2. Berechnen Sie Umfang und Fläche:
   - Umfang: `2 * Math.PI * radius`
   - Fläche: `Math.PI * radius * radius`
3. Deklarieren Sie die Ergebnisvariablen direkt beim Aufruf mit `out double ...` und geben Sie beide Werte aus.

Tests: Bei Radius `1` ist der Umfang etwa `6,283` und die Fläche etwa `3,142`. Bei Radius `0` sind beide Ergebnisse `0`.

Aufgabe 9: Text prüfen und eine Zahl ausgeben

Ziel: Rückgabewert, `out`, Bedingung und `in` kombinieren.

1. Legen Sie einen Text `input` an und wandeln Sie ihn mit `int.TryParse` um. Speichern Sie Erfolg und Zahl getrennt.
2. Bei Erfolg rufen Sie `PrintValue` mit der Zahl auf. Sonst geben Sie `Keine gültige ganze Zahl.` aus.
3. Schreiben Sie dafür `void PrintValue(in int value)`. Die Methode gibt `value` aus. Schreiben Sie `in` auch beim Aufruf.
4. Erklären Sie, warum `value = 6;` im Methodenkörper nicht erlaubt wäre. Was wäre bei einem normalen Parameter bzw. bei `ref` anders?

Tests: `"42"` ergibt `42`, `"0"` ergibt `0`, `"Hallo"` ergibt die Fehlermeldung.

Aufgabe 10: Rekursive Summe berechnen und prüfen

Ziel: Rekursion mit Rückgabewert und Prüfmethode verbinden.

1. Schreiben Sie `int SumUpTo(int n)` für `n >= 1` selbst.
2. Bei `n == 1` liefern Sie `1` zurück. Sonst addieren Sie `n` zum Ergebnis von `SumUpTo(n - 1)`.
3. Speichern Sie das Ergebnis im Hauptprogramm. Prüfen Sie es mit `IsEven` aus Aufgabe 5 und geben Sie Summe sowie `Gerade` oder `Ungerade` aus.
4. Zeichnen Sie für `SumUpTo(3)` die Aufrufe und Rückgaben auf. Markieren Sie den Basisfall.

Tests: `1` ergibt `1` und `Ungerade`, `3` ergibt `6` und `Gerade`, `4` ergibt `10` und `Gerade`.